TECHNISCHE
UNIVERSITÄT
DARMSTADT

# Copula-Calibrated Graph-Conditioned Synthetic Data Generation

Final Technical Report and Developer Continuation Guide

**Nishant Lamboria**

Technische Universität Darmstadt

Department of Electrical Engineering and Information Technology

**Supervisor:** Philipp Fröhlich

July 2026

# Abstract

This report documents the final implementation of a copula-calibrated, graph-conditioned synthetic-data generator for continuous tabular data. The system separates empirical marginal modelling from dependence calibration, stores reusable pair-copula and conditional D-vine mechanisms, and generates observations by traversing a supplied directed acyclic graph in topological order. The final software supports local parent sets of size zero, one, two, and three. One-parent mechanisms use fitted bivariate copulas; two-parent mechanisms use explicit three-dimensional D-vines with optional quantile-binned conditional copulas; three-parent mechanisms use explicit four-dimensional D-vines and select among all six parent permutations by held-out conditional log-likelihood.

The final repository contains calibration configurations for the Sachs protein-signalling data, the scikit-learn diabetes covariates, and a selected subset of Breast Cancer Wisconsin diagnostic covariates. Verified final summaries record 166 pair-copula models, 1,515 two-parent mechanisms, and 4,140 three-parent mechanisms, all with successful status. Controlled validation of the three-parent mechanism comprised 15 runs across sample sizes 500, 1,000, and 2,000. The known generating parent order was recovered in all runs; the overall mean pairwise Kendall-$\tau$ mean absolute error was 0.00703 and the overall mean four-dimensional energy distance was 0.000927. End-to-end generation checks produced 2,000 finite observations for each supported dataset under mixed-indegree DAGs with maximum indegree three.

The report distinguishes statistical graph conditioning from causal identification: the supplied graph determines the simulator factorization, but the software does not identify that graph from observational data and does not implement intervention semantics. In addition to the statistical method, this document specifies the repository architecture, artifact contracts, command-line workflows, validation evidence, limitations, and concrete extension points for future maintainers.

# Contents

Chapter 1

# Problem statement and project objectives

## 1.1 Implemented task

The Copula Causal Simulator constructs synthetic continuous tabular data from two distinct inputs:

1. a real calibration table with variables $X_1, \ldots, X_d$; and
2. a supplied or randomly generated directed acyclic graph (DAG) $G$ on the same ordered variables.

The calibration table determines empirical univariate marginals and reusable local dependence mechanisms. The graph determines which local mechanism is used for each variable during generation. For a graph with parent sets $\mathrm{pa}_G(j)$, the simulator implements the graph-conditioned factorization

$$p_G(x_1, \ldots, x_d) = \prod_{j=1}^{d} p_j\left(x_j \mid x_{\mathrm{pa}_G(j)}\right), \tag{1.1}$$

subject to the final bound $|\mathrm{pa}_G(j)| \leq 3$.

The output consists of a synthetic table in copula coordinates $U \in (0,1)^d$, the same table mapped back to the original empirical marginal scales, the graph, and generation metadata. The package implementation resides under `src/copula_causal_sim`; orchestration and command-line interfaces are under `scripts`.

## 1.2 Why graph-conditioned generation rather than unrestricted density estimation?

An unrestricted multivariate density model would learn a single joint distribution for a fixed set of variables. The project instead needs a reusable mechanism library that can be combined with different DAGs. This changes the engineering problem: calibration must produce local conditional samplers indexed by a child and its parent set. The final design therefore fits:

- empirical marginal models for every variable;
- one bivariate copula for every unordered pair, reused for one-parent mechanisms;
- one explicit three-dimensional D-vine for every child/two-parent combination; and
- one explicit four-dimensional D-vine for every child/three-parent combination.

This decomposition permits a graph to be changed without refitting the raw data, provided every required local mechanism exists in the relevant library.

## 1.3 Final objectives

The accepted implementation satisfies the following objectives.

1. **Separate marginals and dependence.** Empirical inverse marginals preserve observed variable scales, while copulas represent dependence in the unit hypercube.
2. **Support heterogeneous dependence.** Pair-copula family selection uses a finite candidate set and allows rotations.
3. **Support bounded multi-parent mechanisms.** Explicit D-vines provide conditional samplers for two and three parents.
4. **Select parent order from data.** The two-parent fitter compares both orders; the three-parent fitter compares all six permutations using a deterministic train/validation split.
5. **Persist reusable artifacts.** Models are stored as JSON pair-copula files plus pickled library metadata and CSV summaries.
6. **Validate software and statistical behaviour.** Tests cover loading, graph logic, marginal transforms, conditional sampling, serialization, reproducibility, and supported indegrees; controlled simulations evaluate family recovery and the three-parent mechanism.
7. **Provide an operational workflow.** YAML configurations, command-line scripts, checkpointed fitting, graph generation, evaluation, and packaging support reproduction and maintenance.

## 1.4 Scope and non-goals

**Final scope**

The system calibrates continuous observational data and generates from a user-supplied DAG with maximum indegree three. It does not infer a causal graph, estimate interventions, or guarantee that the supplied graph has a causal interpretation.

The word "causal" in the repository name refers to the intended use of graph-structured synthetic data in supervised causal-learning experiments. The graph is an input to the simulator. Equation (1.1) is therefore a modelling factorization, not an identification result. In particular:

- no causal discovery algorithm is implemented;
- no do-operator, structural intervention, or counterfactual query is exposed;
- observational calibration does not establish causal sufficiency or direction; and
- realism is conditional on the supplied graph and the fitted local mechanisms.

Discrete or mixed-type variables are also outside the accepted scope. The loader requires numeric columns, and the final calibration datasets contain continuous variables without missing values.

Chapter 2

# Statistical foundations

This chapter introduces only the concepts used directly by the implementation. The principal code references are `copulas/marginals.py`, `copulas/paircopula_library.py`, `copulas/conditional_vine_library.py`, and `copulas/conditional_vine_indegree3.py` within `src/copula_causal_sim`.

## 2.1 Marginal distributions and copulas

Let $X = (X_1, \ldots, X_d)$ have continuous marginal distribution functions $F_1, \ldots, F_d$. Sklar's theorem states that there exists a copula $C : [0,1]^d \to [0,1]$ such that

$$F(x_1, \ldots, x_d) = C(F_1(x_1), \ldots, F_d(x_d)) \,. \tag{2.1}$$

For continuous marginals, the copula is unique [5, 10]. This separation is the basis of the simulator: calibration maps observations to $U_j = F_j(X_j)$, fits dependence in $U$-space, and finally applies empirical inverse marginals.

## 2.2 Pseudo-observations and empirical marginals

For a calibration table with $n$ rows, the implementation transforms column $j$ by average ranks:

$$U_{ij} = \frac{\operatorname{rank}(X_{ij})}{n+1}. \tag{2.2}$$

The values are clipped to $[\varepsilon, 1-\varepsilon]$, with $\varepsilon = 10^{-6}$ in all final dataset configurations. Division by $n+1$ keeps ordinary ranks strictly inside the unit interval; average ranks give deterministic tie handling. The code is `copulas/marginals.py::make_pseudo_observations`.

Each empirical marginal stores the sorted observed column. Given $u \in (0,1)$, the inverse mapping uses linearly interpolated empirical quantiles through `numpy.quantile`. Consequently, generated original-scale values remain within the observed marginal range, but they are not restricted to the exact empirical support points.

## 2.3 Bivariate copulas and final candidate families

Every bivariate mechanism is selected from the final candidate set

$$\{\text{independence}, \text{Gaussian}, \text{Student}, \text{Clayton}, \text{Gumbel}, \text{Frank}\}. \tag{2.3}$$

Rotations are allowed in the final configurations, permitting asymmetric Archimedean families to represent negative or alternative-tail dependence. The family set is instantiated by `paircopula_library.py::build_fit_controls` and `conditional_vine_library.py::`

`build_bicop_fit_controls`. The YAML files list the same families, although the final code constructs the Python family set explicitly; this duplication is discussed in Chapter 12.

The final dataset configurations use the Bayesian information criterion

$$\mathrm{BIC} = -2\ell(\widehat{\theta}) + k \log n, \tag{2.4}$$

where $\ell$ is the fitted log-likelihood and $k$ the effective number of parameters [7]. The command-line builder also exposes AIC and mBIC choices, but those were not used for the final real-data libraries. Bivariate fitting and inverse conditional transforms are delegated to `pyvinecopulib` [13].

## 2.4 Kendall's rank dependence

Kendall's $\tau$ is used as a rank-based dependence diagnostic [4]. For continuous observations it can be written as

$$\tau = \Pr\big[(X - X')(Y - Y') > 0\big] - \Pr\big[(X - X')(Y - Y') < 0\big], \tag{2.5}$$

where $(X', Y')$ is an independent copy. Rank diagnostics are appropriate here because copulas are invariant under strictly increasing marginal transformations. The final evaluation reports pairwise Kendall-matrix error in controlled three-parent validation, and several calibration summaries store empirical Kendall and Spearman values.

## 2.5 Conditional copulas and h-functions

For a continuous bivariate copula $C(u, v)$, this report uses the orientation

$$h_1(u, v) = \frac{\partial C(u, v)}{\partial u} = F_{V|U=u}(v), \tag{2.6}$$

$$h_2(u, v) = \frac{\partial C(u, v)}{\partial v} = F_{U|V=v}(u). \tag{2.7}$$

Thus $h_1^{-1}(u, q)$ inverts with respect to the second coordinate and samples $V \mid U = u$ from $q \sim \mathrm{Unif}(0, 1)$. The implementation explicitly distinguishes `hfunc1`/`hinv1` from `hfunc2`/`hinv2`, because the stored pair order determines which conditional distribution is required. The final orientation follows the `pyvinecopulib.Bicop` API and is tested in the generator and conditional-vine test modules.

## 2.6 Pair-copula constructions and D-vines

Pair-copula constructions factor a multivariate copula density into bivariate copula densities, some evaluated on conditional distribution coordinates [1–3]. The project uses explicit D-vines rather than a general R-vine data structure because each mechanism has a fixed small dimension and a distinguished child.

For a two-parent mechanism with order $P_1 - P_2 - X$, the three-dimensional copula density is

$$c(u_1, u_2, u_x) = c_{12}(u_1, u_2) c_{2x}(u_2, u_x) c_{1x;2}\Big(u_{1|2}, u_{x|2}\Big), \tag{2.8}$$

with $u_{1|2} = h_2^{12}(u_1, u_2)$ and $u_{x|2} = h_1^{2x}(u_2, u_x)$.

For a three-parent mechanism with order $P_1 - P_2 - P_3 - X$, the final four-dimensional construction contains six pair copulas:

$$c(u_1, u_2, u_3, u_x) = c_{12}c_{23}c_{3x}c_{13;2}c_{2x;3}c_{1x;23}, \tag{2.9}$$

where each higher-tree factor is evaluated on the corresponding h-function coordinates. The child is always placed last; the fitter evaluates all six permutations of the three parents.

## 2.7 Connection to DAG factorization

The D-vine is local: it represents $p(x_j \mid x_{\mathrm{pa}(j)})$ for one child and one parent set. The global generator combines these local conditionals according to a topological ordering of the supplied DAG. This is not the same as fitting one unrestricted $d$-dimensional vine. It is a mechanism library indexed by graph-local queries, which makes graph reuse possible but also means omitted graph edges deliberately remove dependencies from the generated factorization.

Chapter 3

# Data and preprocessing

## 3.1 Final calibration datasets

The final repository snapshot contains three numeric calibration tables. Their dimensions were recalculated directly from `data/sachs.data.txt`, `data/diabetes.csv`, and `data/breast_cancer.csv`; all contain zero missing entries.

**Table 3.1:** Final calibration data. Counts are derived from the files under `data/`.

| Dataset | Rows | Variables | Missing | Role |
|---|---|---|---|---|
| Sachs | 853 | 11 | 0 | protein-signalling calibration |
| Diabetes | 442 | 10 | 0 | continuous covariate calibration |
| Breast Cancer | 569 | 12 | 0 | selected diagnostic covariates |

### 3.1.1 Sachs protein-signalling data

The Sachs table contains 853 observations of 11 protein and phospholipid measurements: Raf, Mek, Plcg, PIP2, PIP3, Erk, Akt, PKA, PKC, P38, and Jnk. The data originate from the single-cell signalling study of Sachs et al. [6]. The repository loads the whitespace-delimited file directly through `configs/datasets/sachs.yaml`. In this project the table is used only as continuous observational calibration data; the simulator does not recover or assert the biological network from the table.

### 3.1.2 Diabetes covariates

The diabetes table contains 442 rows and the ten baseline features returned by `sklearn.datasets.load_diabetes`: age, sex, body-mass index, blood pressure, and six serum measurements [9]. The preparation script is `scripts/prepare_sklearn_diabetes.py`. The disease-progression target is excluded, because the project calibrates a joint distribution over continuous features rather than a supervised response model.

### 3.1.3 Breast Cancer Wisconsin diagnostic covariates

The source dataset contains 569 observations and 30 real-valued diagnostic features [8, 11]. The final preparation script, `scripts/prepare_sklearn_breast_cancer.py`, retains 12 predictors: mean radius, mean texture, mean smoothness, mean compactness, mean concavity, mean concave points, mean symmetry, mean fractal dimension, radius error, texture error, worst radius, and worst texture. The diagnosis target is excluded. The selected-column provenance is also recorded in `data/breast_cancer_metadata.json`.

## 3.2 Loader contract

`data/loaders.py::load_tabular_dataset` resolves the path and file type from YAML, selects configured continuous columns, drops configured columns, and applies the missing-value policy. The final configurations specify:

- `continuous_columns: all`;
- `missing_policy: error`;
- no dropped columns; and
- `clip_eps: 1e-6`.

The loader rejects non-numeric data after selection. This establishes a clear boundary: preprocessing that creates a continuous numeric table occurs before copula calibration.

## 3.3 Four representations of the data

The software uses four related but distinct objects.

1. **Original calibration data $X$.** Values are on the dataset's observed scales.
2. **Calibration pseudo-observations $U$.** Equation (2.2) maps each column to ranks in the unit hypercube. These are saved as `pseudo_observations.csv` in each artifact directory.
3. **Generated copula-space data $\widetilde{U}$.** The graph-conditioned sampler returns values in $[\varepsilon, 1-\varepsilon]^d$.
4. **Generated original-scale data $\widetilde{X}$.** Empirical inverse marginals transform each column of $\widetilde{U}$ back to its calibration scale.

The dependence models operate only on $U$-coordinates. This prevents large-scale variables from dominating fitting and allows the same conditional-sampling logic to be used across all datasets.

## 3.4 Column identity and order

Column order is a system-wide data contract. A graph adjacency matrix has one row and column per dataset variable in exactly the order stored by the marginal and copula libraries. The generators reject graph dimensions that do not equal the number of library columns. They also require input $U$ dataframes passed to the marginal inverse transform to have exactly the stored column sequence. This guards against silent relabelling of graph nodes or generated variables.

Chapter 4

# Final system architecture

## 4.1 End-to-end pipeline

Figure 4.1 summarizes the final design. Calibration is performed once per dataset; generation can then be repeated for any compatible DAG. The boundary between the two phases is the artifact library.

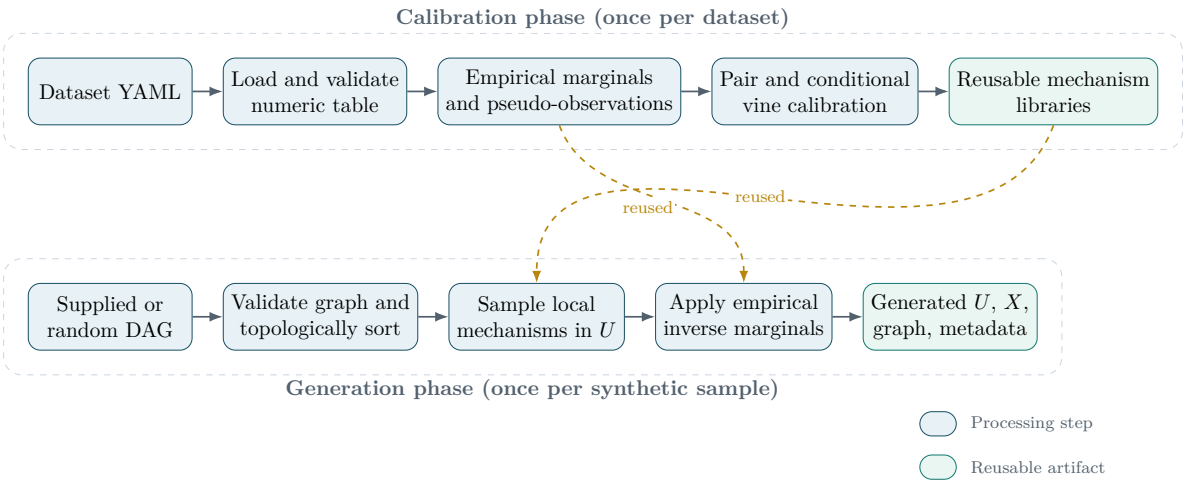



**Figure 4.1:** Final architecture. Calibration constructs reusable local mechanisms; generation combines them according to a validated DAG. Dashed amber arrows indicate calibration outputs consumed during generation.

## 4.2 Package layers

The reusable package is organized by responsibility:

| | |
|---|---|
| `data/` | Dataset loading and numeric validation. |
| `copulas/` | Empirical marginals, pair-copula calibration, explicit two-parent D-vines, adaptive conditional bins, and explicit three-parent D-vines. |
| `graphs/` | Adjacency validation, degree queries, topological sorting, and bounded-indegree random DAG sampling. |
| `generators/` | Graph-conditioned sampling for maximum indegree one, two, and three. |
| `evaluation/` | Marginal, rank-correlation, graph-pair, and empirical-tail diagnostics. |
| `synthetic/` | Controlled pair-family and complete-mechanism validation utilities. |

The `scripts/` directory contains thin orchestration CLIs around these package components.

## 4.3 Persistent artifacts

Each dataset configuration specifies an artifact directory and a table directory. The principal artifact contract is:

- `marginals.pkl`: empirical marginal library;
- `pseudo_observations.csv`: ranked calibration data;
- `coupling_library.pkl`: pair-copula metadata;
- `paircopula_models/*.json`: serialized bivariate copulas;
- `conditional_vine_library_indegree2.pkl` and its JSON mechanism directory;
- `conditional_vine_library_indegree3.pkl` and its JSON mechanism directory;
- CSV summaries in `reports/tables/<dataset>/`.

The three exhaustive indegree-three libraries are additionally packaged as compressed Git-LFS archives with SHA-256 manifests under `artifacts/precomputed_indegree3/`. The report bundle includes the manifests and LFS listing but not the large archives themselves.

## 4.4 Graph and metadata contracts

An adjacency matrix is a headerless square CSV with the convention

$$A_{ij} = 1 \quad \Longleftrightarrow \quad i \to j.$$

`graphs/dag.py` enforces binary entries, zero diagonal, squareness, acyclicity, and an optional maximum indegree. Kahn's algorithm supplies the topological order. Generation metadata records the dataset identifier, seed, sample count, graph summary, and artifact paths. The captured end-to-end metadata uses absolute Windows paths; these are informative provenance fields rather than portable lookup keys, and this is recorded as a software limitation.

## 4.5 Randomness and clipping

Graph sampling and data generation use `numpy.random.default_rng` with explicit integer seeds. Root variables are sampled independently from $\mathrm{Unif}(\varepsilon, 1-\varepsilon)$. All conditional outputs are clipped to the same interval after inverse h-function evaluation. The clip is a numerical-domain safeguard, not a fitted tail model.

Chapter 5

# Construction of reusable calibration libraries

## 5.1 Empirical marginals and pair library

`scripts/build_coupling_library.py` performs the base calibration. It loads the table, writes a dataset summary, creates pseudo-observations, fits one empirical marginal per column, and fits one bivariate copula for every unordered variable pair. For $d$ variables, this produces $\binom{d}{2}$ records. Each pair record stores variable order, selected family, rotation, parameters, log-likelihood, information criteria, rank diagnostics, tail diagnostics, status, and a JSON model path.

The final pair-family counts are shown in Table 5.1. These counts are descriptive of the selected real-data libraries, not evidence that one family is universally preferable.

**Table 5.1:** Selected family counts in the final pair-copula summaries. Source: `reports/tables/<dataset>/paircopula_summary.csv`.

| Dataset | Indep. | Gaussian | Student | Clayton | Gumbel | Frank |
|---|---|---|---|---|---|---|
| Sachs | 46 | 0 | 2 | 3 | 3 | 1 |
| Diabetes | 3 | 17 | 1 | 5 | 3 | 16 |
| Breast Cancer | 8 | 10 | 1 | 13 | 18 | 16 |

## 5.2 One-parent mechanisms

A one-parent query $(P \to X)$ is resolved through the unordered pair library. The record also retains the order used during fitting. If the fitted JSON model is ordered as $(P, X)$, the generator draws $Q \sim \mathrm{Unif}(0, 1)$ and computes

$$U_X = \left(h_1^{PX}\right)^{-1}(U_P, Q). \tag{5.1}$$

If the fitted order is $(X, P)$, the corresponding `hinv2` orientation is used. This explicit branch prevents a common conditional-direction error: retrieving the correct unordered pair is not enough; the inverse h-function must match the stored coordinate order.

## 5.3 Two-parent D-vine mechanisms

For each child $X$ and unordered parent set $\{P_a, P_b\}$, the builder evaluates both orders $P_a - P_b - X$ and $P_b - P_a - X$. A deterministic split uses the configured validation fraction and seed. For an order $P_1 - P_2 - X$, the fit uses the three factors in Equation (2.8). The held-out order

score excludes the parent-only factor and averages

$$\log c_{2x}(u_2, u_x) + \log c_{1x;2}(u_{1|2}, u_{x|2}), \tag{5.2}$$

which is the part relevant to $X \mid P_1, P_2$. If the requested held-out split cannot satisfy the minimum validation size, the implementation falls back to negative conditional BIC. The selected order is then refitted on all observations.

### 5.3.1 Adaptive quantile-binned conditional edge

The final two-parent implementation can relax the simplifying assumption for the last conditional pair $C_{1x;2}$. It first fits a simplified candidate, then considers equal-frequency bins of the conditioning coordinate $U_2$. Candidate bin counts are 1 through 6; every bin must contain at least 75 training observations. Family and rotation are fixed across bins while parameters may vary. A multi-bin model is selected only when its mean held-out conditional log-likelihood exceeds the simplified score by at least 0.005.

**Table 5.2:** Final two-parent mechanism types. "Quantile-binned" means more than one conditional bin was selected. Source: final indegree-two summary CSVs.

| Dataset | Simplified | Quantile-binned | Selected bins > 1 | Total |
|---|---|---|---|---|
| Sachs | 471 | 24 | 24 | 495 |
| Diabetes | 274 | 86 | 86 | 360 |
| Breast Cancer | 406 | 254 | 254 | 660 |

The adaptive model is used only for two-parent mechanisms. The accepted three-parent implementation remains simplified at every conditional edge.

## 5.4 Three-parent D-vine mechanisms

For each child $X$ and unordered parent set $S = \{P_a, P_b, P_c\}$, `conditional_vine_indegree3.py` enumerates all $3! = 6$ parent permutations. With order $P_1 - P_2 - P_3 - X$, the first tree contains $C_{12}$, $C_{23}$, and $C_{3x}$; the second tree contains $C_{13;2}$ and $C_{2x;3}$; the third contains $C_{1x;23}$.

The held-out conditional score is

$$\frac{1}{n_{\text{val}}} \sum_{i \in \text{val}} \left[ \log c_{3x}^{(i)} + \log c_{2x;3}^{(i)} + \log c_{1x;23}^{(i)} \right]. \tag{5.3}$$

Parent-only factors $c_{12}$, $c_{23}$, and $c_{13;2}$ are deliberately excluded from order comparison: the criterion targets the conditional law of the child rather than the fit of the parent distribution. Candidate statuses, scores, and errors are retained in the summary record. The best order is refitted on all data and serialized as six pair-copula JSON files.

The lookup key is $(X, \text{frozenset}(S))$. Thus graph parent order does not affect retrieval; the selected D-vine order stored inside the record controls sampling.

## 5.5 Exhaustive library sizes

For $d$ variables, exhaustive counts are

$$N_1(d) = \binom{d}{2}, \tag{5.4}$$

$$N_2(d) = d\binom{d-1}{2}, \tag{5.5}$$

$$N_3(d) = d\binom{d-1}{3}. \tag{5.6}$$

The first count is unordered because one pair record serves both orientations; the multi-parent counts distinguish the child. Table 5.3 reports the verified final summaries.

**Table 5.3:** Final library record counts and successful statuses. "Pair" denotes unordered bivariate records; multi-parent counts distinguish the child.

| Dataset | Pair | Pair OK | Two-parent | Two-parent OK | Three-parent | Three-parent OK |
|---|---|---|---|---|---|---|
| Sachs | 55 | 55 | 495 | 495 | 1320 | 1320 |
| Diabetes | 45 | 45 | 360 | 360 | 840 | 840 |
| Breast Cancer | 66 | 66 | 660 | 660 | 1980 | 1980 |
| Total | 166 | 166 | 1515 | 1515 | 4140 | 4140 |

The 4,140 three-parent records reference 24,840 JSON pair-copula models in total. This is a derived count from the three artifact manifests: each four-dimensional D-vine contains six pair copulas.

## 5.6 Parallel fitting and resumability

The exhaustive three-parent task is embarrassingly parallel across child/parent-set combinations. `scripts/run_indegree3_parallel.py` uses a `ProcessPoolExecutor`; each worker loads pseudo-observations once, uses one internal numerical thread, and fits independent mechanisms. A checkpoint pickle stores records keyed by canonical parent set. Atomic replacement of a temporary checkpoint reduces corruption risk. Running without `-restart` resumes missing mechanisms; `-retry-failed` removes failed records from the checkpoint and refits them.

This parallel runner changes orchestration, not the statistical estimator. It calls the same production fitting function as the serial builder and emits the same library class and summary schema.

## 5.7 Failure handling and auditability

Fit functions return structured records with `status` and `error` fields. Candidate-order errors are retained separately so one failed ordering need not obscure successful alternatives. Final library counts in Table 5.3 are based on summary status fields; the report does not infer success merely from file existence.

Chapter 6

# Graph-conditioned synthetic generation

## 6.1 Generation algorithm

The maximum-indegree-three generator composes the lower-order generators. It validates the graph, allocates an $n \times d$ matrix of missing values, and visits nodes in topological order. A node's parent count determines the local sampler. After all columns are generated, finite-value checks and clipping are applied, followed by empirical inverse marginals.

**Input:** DAG adjacency $A$; sample count $n$; seed $s$; marginal, pair, two-parent and three-parent libraries
**Output:** Copula-space table $\widetilde{U}$ and original-scale table $\widetilde{X}$
Validate that $A$ is square, binary, loop-free, acyclic, and has maximum indegree at most three;
Check that the graph dimension equals the stored variable count;
Initialize a seeded random-number generator and an empty $n \times d$ matrix $\widetilde{U}$;
**foreach** *node $j$ in a topological order of $A$* **do**
  $P \leftarrow \mathrm{pa}_A(j)$;
  **if** $|P| = 0$ **then**
    sample $\widetilde{U}_j \sim \mathrm{Unif}(\varepsilon, 1-\varepsilon)$
  **else**
    **if** $|P| = 1$ **then**
      retrieve pair record and apply the orientation-correct inverse h-function
    **else if** $|P| = 2$ **then**
      retrieve the two-parent D-vine and execute the inverse D-vine chain
    **else if** $|P| = 3$ **then**
      retrieve the three-parent D-vine and execute the inverse D-vine chain
    **end**
  **end**
**end**
Reject non-finite or missing values and clip to $[\varepsilon, 1-\varepsilon]$;
$\widetilde{X} \leftarrow$ columnwise empirical inverse marginal transform of $\widetilde{U}$;
**return** $(\widetilde{U}, \widetilde{X}, A, s)$;

**Algorithm 1.** Graph-conditioned generation with maximum indegree three

## 6.2 Root and one-parent nodes

Roots are independent uniforms in copula space. This follows directly from the factorized model: without a parent edge, no calibrated dependency is introduced. A one-parent node uses the bivariate pair record as described in Chapter 5. The generator verifies that the record variables match the requested parent-child pair and selects `hinv1` or `hinv2` accordingly.

## 6.3 Two-parent inverse D-vine chain

For stored order $P_1 - P_2 - X$, let

$$W_1 = h_2^{12}(U_1, U_2) = F_{P_1|P_2}(P_1 \mid P_2). \tag{6.1}$$

Draw $Q \sim \mathrm{Unif}(0,1)$. For a simplified mechanism,

$$W_X = \left(h_1^{1x;2}\right)^{-1}(W_1, Q), \tag{6.2}$$

$$U_X = \left(h_1^{2x}\right)^{-1}(U_2, W_X). \tag{6.3}$$

For a quantile-binned mechanism, the first inverse in Equation (6.3) uses the bin model selected by $U_2$; the second inverse is unchanged. This is implemented in `generators/indegree_two.py`.

## 6.4 Three-parent inverse D-vine chain

For stored order $P_1 - P_2 - P_3 - X$, the parent-side transforms are

$$W_{1|2} = h_2^{12}(U_1, U_2), \tag{6.4}$$

$$W_{3|2} = h_1^{23}(U_2, U_3), \tag{6.5}$$

$$W_{1|23} = h_2^{13;2}(W_{1|2}, W_{3|2}), \tag{6.6}$$

$$W_{2|3} = h_2^{23}(U_2, U_3). \tag{6.7}$$

Then, for $Q \sim \mathrm{Unif}(0,1)$, the child is generated by three inverse h-functions:

$$W_{X|23} = \left(h_1^{1x;23}\right)^{-1}(W_{1|23}, Q), \tag{6.8}$$

$$W_{X|3} = \left(h_1^{2x;3}\right)^{-1}(W_{2|3}, W_{X|23}), \tag{6.9}$$

$$U_X = \left(h_1^{3x}\right)^{-1}(U_3, W_{X|3}). \tag{6.10}$$

These equations match `conditional_vine_indegree3.py::sample_child_from_fitted_dvine4` and `generators/indegree_three.py::_sample_child_given_three_parents`. Explicit intermediate coordinates make the orientation auditable and testable.

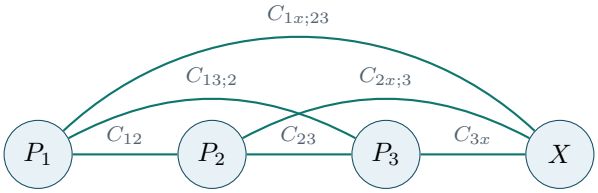



**Figure 6.1:** Explicit four-dimensional D-vine used for a three-parent mechanism. The child is fixed last; all six parent orders are candidates during calibration.

## 6.5 Output persistence

`scripts/generate_from_graph_indegree3.py` writes `generated_u.csv`, `generated_x.csv`, an adjacency copy, and `generation_metadata.json`. The result object also retains the column sequence, seed, and maximum supported indegree. The original-scale output is suitable for downstream experiments that expect calibration-like column scales; the copula-space output is useful for dependence diagnostics.

## 6.6 What the graph controls

The graph controls which variables are conditioned on and therefore which calibrated mechanisms are activated. It does not force the simulator to reproduce all dependencies of the original calibration distribution. For example, two graph roots are sampled independently even if they were dependent in the source table. Fidelity claims must therefore distinguish a controlled local-mechanism validation from an arbitrary-graph functional demonstration.

Chapter 7

# Software implementation and user workflow

## 7.1 Environment setup

The captured final environment used Python 3.11.9. The principal recorded package versions are listed in Table 7.1; the complete freeze is preserved in `generated/data/metadata.json` within this report project and in `_project_metadata/pip_freeze.txt` in the repository bundle.

**Table 7.1:** Principal package versions captured with the final repository bundle.

| Component | Captured version |
|---|---|
| Python | 3.11.9 |
| NumPy | 2.4.6 |
| pandas | 3.0.3 |
| SciPy | 1.17.1 |
| pyvinecopulib | 0.7.6 |
| scikit-learn | 1.9.0 |
| Matplotlib | 3.10.9 |
| pytest | 9.0.3 |
| PyYAML | 6.0.3 |

A Windows setup is:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
$env:PYTHONPATH = "$PWD\src"
```

The scripts also insert `src/` into `sys.path`, so the explicit environment variable is primarily useful for direct package imports and test invocations.

## 7.2 Configuration

Each YAML file under `configs/datasets/` defines the dataset identifier, input path and file type, data policy, copula controls, and output directories. The final configurations use BIC, rotations, a 20% order-validation split, order-selection seed 42, minimum validation size 50, adaptive quantile bins for two-parent mechanisms, minimum bin size 75, and minimum score improvement 0.005.

## 7.3 Calibration commands

The base coupling library is built with the only required flag exposed by the script:

```
python scripts/build_coupling_library.py \
  --config configs/datasets/sachs.yaml
```

Two-parent mechanisms are built with:

```
python scripts/build_conditional_vine_library.py \
  --config configs/datasets/sachs.yaml \
  --parent-set-size 2
```

The same serial builder accepts `-parent-set-size 3`, but the exhaustive final libraries were practically constructed with the resumable multiprocessing runner:

```
python -u scripts/run_indegree3_parallel.py \
  --config configs/datasets/sachs.yaml \
  --workers 8 \
  --checkpoint-every 1 \
  --restart
```

To resume, rerun without `-restart`. The runner also exposes `-limit` for smoke tests and `-retry-failed`.

## 7.4 Graph commands

Random bounded-indegree DAGs are generated by:

```
python scripts/generate_random_graphs.py \
  --num-graphs 5 \
  --num-nodes 11 \
  --edge-prob 0.25 \
  --max-indegree 3 \
  --seed 42 \
  --output-dir artifacts/graphs/example
```

The implementation first samples a random topological ordering and considers edges only from earlier to later nodes, so acyclicity is structural rather than repaired after sampling.

## 7.5 Synthetic generation command

The verified indegree-three CLI is:

```
python scripts/generate_from_graph_indegree3.py \
  --config configs/datasets/sachs.yaml \
  --graph graphs/end_to_end/sachs_mixed_indegree.csv \
  --n-samples 2000 \
  --seed 42 \
  --output-dir outputs/end_to_end/sachs_seed42
```

Separate scripts exist for maximum indegree one and two. The indegree-three generator can itself process mixed graphs containing zero-, one-, two-, and three-parent nodes.

## 7.6 Evaluation command

`scripts/evaluate_generated_data.py` requires real-data configuration, generated original-scale CSV, and output directory; a graph is optional:

```
python scripts/evaluate_generated_data.py \
  --config configs/datasets/sachs.yaml \
  --generated-x outputs/example/generated_x.csv \
  --graph graphs/example/graph_0000_adjacency.csv \
  --output-dir reports/evaluation/example \
  --tail-probability 0.05
```

It writes marginal-quantile, Pearson, Spearman, Kendall, graph-pair, and empirical-tail tables and summaries. These generic capabilities are documented here; only final result files actually present in the bundle are reported numerically in Chapter 10.

## 7.7 Packaging precomputed libraries

`scripts/package_indegree3_artifacts.py` validates the three final summaries, creates one compressed archive per dataset, writes manifests, and records SHA-256 hashes. Git LFS tracks the archives. Another checkout must execute `git lfs pull` before extraction.

## 7.8 Tests and final checks

The complete suite is run with:

```
python -m pytest -q
```

The captured final output records 175 passing tests and exit code zero. The repository also contains `scripts/validate_final_indegree3_release.py`, which checks final artifacts and experiment files. In the supplied bundle its captured invocation failed before execution because the package import path was not configured; this report therefore relies on the successful pytest record and independent file-level audit rather than claiming a successful release-script run.

Chapter 8

# Validation and testing

## 8.1 Validation layers

The final evidence separates three questions:

1. **Can the bivariate selection layer recover known dependence?**
2. **Can the four-dimensional mechanism select parent order and reproduce a controlled joint distribution?**
3. **Can the complete software load saved artifacts and generate finite tables for mixed-indegree graphs?**

These are different claims. Pair-family recovery does not validate the full graph generator, and an end-to-end smoke graph does not by itself establish distributional fidelity to the calibration table.

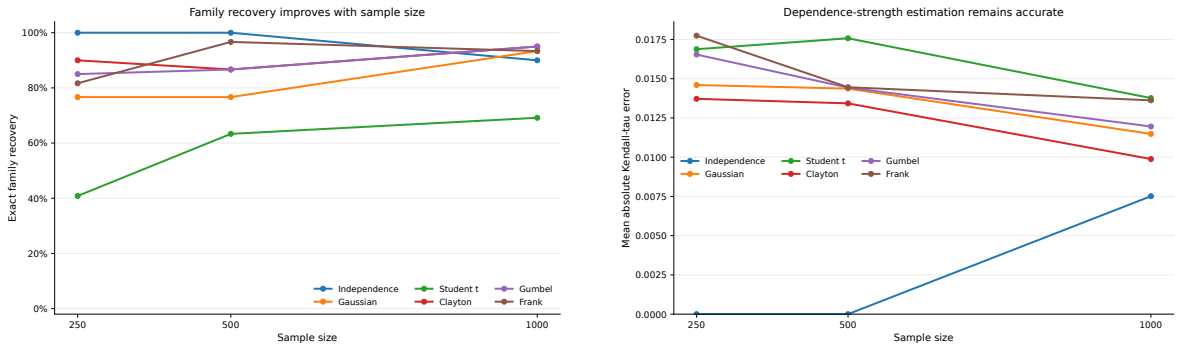
## 8.2 Bivariate family-recovery experiment

The final pair-family report contains 1,140 successful controlled runs covering the six candidate families, multiple dependence strengths, sample sizes 250, 500, and 1,000, and Student degrees of freedom 4 and 10. Each fitted model is selected by the same production controls used by the library code. Exact-family recovery and absolute Kendall-$\tau$ error are then compared with the known generating copula.

**Table 8.1:** Headline pair-family recovery results from `reports/synthetic_validation/pair_family_recovery/headline_statistics.json`.

| Metric | Value |
|---|---|
| Successful simulation runs | 1140 |
| Overall exact-family recovery | 78.9% |
| Overall Kendall $\tau$ MAE | 0.0140 |
| Exact-family recovery at $n = 1000$ | 86.1% |
| Kendall $\tau$ MAE at $n = 1000$ | 0.0122 |
| Student recovery, $\nu = 4$ | 83.3% |
| Student recovery, $\nu = 10$ | 32.2% |

Exact family labels are not always identifiable from moderate samples, especially when Student dependence with large degrees of freedom resembles a Gaussian copula. This is visible in the Student $\nu = 10$ recovery rate. At the same time, the overall Kendall-$\tau$ MAE remains small, showing that dependence-strength recovery can be more stable than exact family classification.




**(a)** Exact-family recovery.

**(b)** Absolute Kendall-$\tau$ error.

**Figure 8.1:** Bivariate recovery diagnostics generated from the final simulation table. Error bars and grouping follow the report-generation code in `scripts/synthetic_validation/analyze_pair_recovery.py`.

## 8.3 Controlled three-parent validation

`scripts/synthetic_validation/run_indegree3_validation.py` generates data from a known four-dimensional D-vine with order $P_1 - P_2 - P_3 - X$. It fits the production three-parent calibrator, scores all six parent permutations, samples the child conditionally on held-out parents, and reports:

- selected order and best-minus-second-best score margin;
- off-diagonal pairwise Kendall-matrix MAE;
- four-dimensional energy distance;
- finite-value and unit-interval checks.

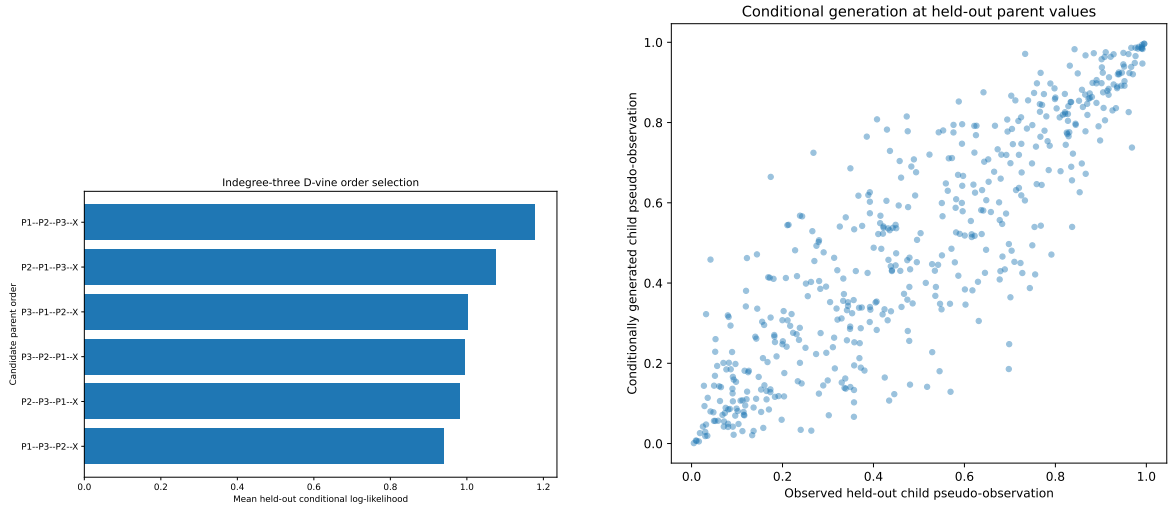
The script rejects sample sizes below 400. The final experiment therefore uses $n \in \{500, 1000, 2000\}$ and seeds 1 through 5, for 15 runs. The aggregation is recorded in `reports/final_validation/indegree3_multiseed/`.

## 8.4 End-to-end functional demonstrations

For each calibration dataset, the final outputs include a deliberately mixed graph with roots and nodes of indegree one, two, and three. Each run generated 2,000 rows with seed 42. Independent audit of the CSV files confirmed finite $U$ and $X$, zero missing values, and copula coordinates within the unit interval.

**Table 8.2:** Verified final end-to-end outputs. These are functional integration demonstrations, not same-graph fidelity benchmarks.

| Dataset | Rows | Variables | Edges | Max indegree | Finite $U, X$ | Missing |
|---|---|---|---|---|---|---|
| Sachs | 2000 | 11 | 11 | 3 | yes | 0 |
| Diabetes | 2000 | 10 | 10 | 3 | yes | 0 |
| Breast Cancer | 2000 | 12 | 12 | 3 | yes | 0 |




**(a)** One representative six-order comparison.

**(b)** Held-out true and generated child behaviour.

**Figure 8.2:** Representative compact validation output. These plots illustrate one run; aggregate conclusions use all 15 final runs.

## 8.5 Software tests

The test tree contains dedicated modules for data loading, graph validation, empirical marginals, pair generation, one-, two-, and three-parent generators, explicit conditional D-vines, adaptive non-simplified conditional models, order selection, complete synthetic validation, recovery reporting, serialization, and reproducibility. The captured final test command completed with 175 passes and no failures.

This count establishes that the committed tests passed in the captured environment; it does not imply exhaustive correctness. Important remaining gaps include fresh-clone artifact installation and systematic cross-platform numerical equivalence.

Chapter 9

# Evaluation methodology

## 9.1 Pair-family recovery rate

For controlled bivariate runs with known generating family $f_i$ and selected family $\widehat{f}_i$, exact recovery is

$$\widehat{R} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\widehat{f}_i = f_i\}. \tag{9.1}$$

Higher is better. This metric is deliberately strict: two families can represent nearly indistinguishable dependence at a given sample size while receiving different labels.

## 9.2 Kendall-$\tau$ error

The bivariate experiment uses $|\widehat{\tau}_i - \tau_i|$. The three-parent experiment compares the empirical $4 \times 4$ Kendall matrices of held-out true and generated samples and averages absolute errors over off-diagonal entries:

$$\mathrm{MAE}_\tau = \frac{1}{d(d-1)}\sum_{j\neq k}\left|\widehat{\tau}_{jk}^{\mathrm{true}} - \widehat{\tau}_{jk}^{\mathrm{gen}}\right|, \qquad d = 4. \tag{9.2}$$

Lower is better. This captures pairwise rank structure but not every higher-order feature.

## 9.3 Energy distance

For samples $x_1, \ldots, x_n \sim P$ and $y_1, \ldots, y_m \sim Q$, the empirical energy distance used by the validation code is

$$\widehat{\mathrm{ED}}(P, Q) = \frac{2}{nm}\sum_{i,j}\|x_i - y_j\| - \frac{1}{n^2}\sum_{i,i'}\|x_i - x_{i'}\| - \frac{1}{m^2}\sum_{j,j'}\|y_j - y_{j'}\|. \tag{9.3}$$

Energy distance is a multivariate distributional discrepancy based on Euclidean distances [12]. Lower is better. In this project it is computed in four-dimensional copula space for the controlled three-parent validation.

## 9.4 Order recovery and score margin

Order recovery is the indicator that the selected parent order equals the known generating order. The score margin is

$$\Delta = S_{(1)} - S_{(2)}, \tag{9.4}$$

where $S_{(1)}$ and $S_{(2)}$ are the best and second-best held-out conditional log-likelihood scores. A positive margin means the selected order was strictly preferred under the scoring rule; its magnitude is not calibrated as a probability.

## 9.5 Finite-output and domain rates

A run passes the finite-output check when every generated coordinate is finite. It passes the unit-interval check when all generated copula coordinates lie in $[0, 1]$; production code actually clips to $[\varepsilon, 1 - \varepsilon]$. Rates are the fraction of runs satisfying each condition. These are numerical validity checks, not distributional-quality metrics.

## 9.6 Generic repository evaluation metrics

`evaluation/realism.py` and `scripts/evaluate_generated_data.py` additionally implement:

- absolute errors of configurable marginal quantiles;
- Pearson, Spearman, and Kendall matrix absolute errors;
- largest pairwise dependence discrepancies;
- edge and non-edge summaries when a graph is supplied; and
- empirical lower- and upper-tail dependence at a configured tail probability, default 0.05.

These metrics form an evaluation API. Numerical values are not reported here unless the corresponding final outputs are present in the supplied repository snapshot.

## 9.7 Experimental design and aggregation

The final three-parent table aggregates five seeds independently at each of three sample sizes. Reported score margins, Kendall MAEs, and energy distances are means plus sample standard deviations over those five runs. Recovery and validity entries are rates. The overall summary averages run-level errors across all 15 runs and reports the median run-level score margin.

The $n = 250$ setting is not part of the accepted experiment because the validation script requires at least 400 observations. No preliminary $n = 250$ output is combined with the final aggregates.

Chapter 10

# Final results

## 10.1 Calibration-library completeness

All final pair, two-parent, and three-parent summary records have status `ok`; the verified counts are given in Table 5.3. Across the three datasets, the final snapshot therefore contains 166 pair records, 1,515 child/two-parent records, and 4,140 child/three-parent records. The three-parent manifests list 7,920, 5,040, and 11,880 JSON pair-copula files for Sachs, Diabetes, and Breast Cancer, respectively.

The two-parent adaptive procedure selected multi-bin conditional models for 24 of 495 Sachs mechanisms, 86 of 360 Diabetes mechanisms, and 254 of 660 Breast Cancer mechanisms. This indicates that the final software exercised both the simplified and non-simplified code paths; it does not establish that quantile binning is the uniquely correct non-simplified model.

## 10.2 Bivariate calibration layer

The controlled pair-family experiment achieved 78.9% exact-family recovery over 1,140 successful runs and 86.1% at $n = 1000$. Overall Kendall-$\tau$ MAE was 0.0140. The contrast between Student $\nu = 4$ recovery (83.3%) and Student $\nu = 10$ recovery (32.2%) is consistent with the statistical difficulty of distinguishing high-degree Student dependence from a Gaussian copula in finite samples. Accordingly, downstream interpretation should emphasize conditional-sampling quality and rank dependence rather than treat every family label as ground truth.

## 10.3 Three-parent order selection and fidelity

**Table 10.1:** Final controlled three-parent validation. Values are mean ± sample standard deviation over five seeds. All runs scored all six parent orders, produced finite values, and remained inside the unit interval.

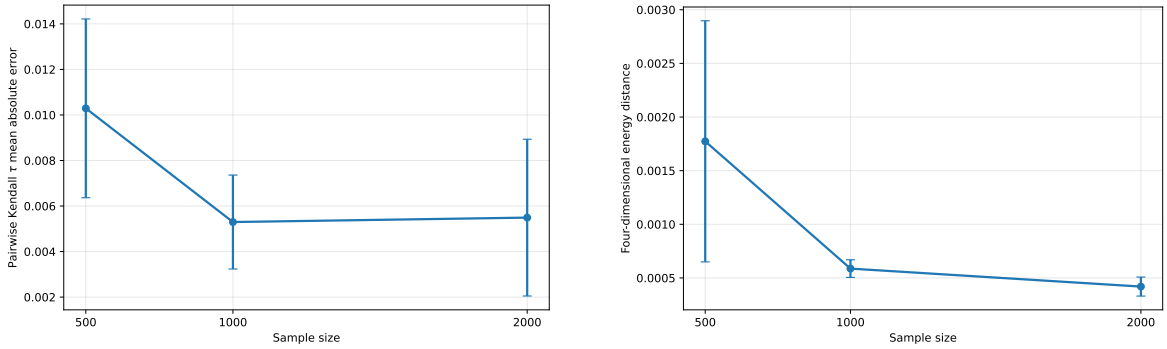
| $n$ | Runs | Recovery | Score margin | Kendall $\tau$ MAE | Energy distance |
|---|---|---|---|---|---|
| 500 | 5 | 100% | $0.1163 \pm 0.0241$ | $0.01029 \pm 0.00393$ | $0.001773 \pm 0.001124$ |
| 1000 | 5 | 100% | $0.0654 \pm 0.0158$ | $0.00530 \pm 0.00206$ | $0.000587 \pm 0.000082$ |
| 2000 | 5 | 100% | $0.0937 \pm 0.0373$ | $0.00549 \pm 0.00344$ | $0.000420 \pm 0.000089$ |

The generating order was recovered in all 15 runs. The mean score margin was positive at every sample size, although it was not monotone in $n$. This is expected for a finite five-seed experiment: the margin depends on the realized sample and on similarity among candidate order factorizations. The result supports reliable order selection in this controlled mechanism, but it is not a universal order-identifiability theorem.

Pairwise Kendall error fell from 0.01029 at $n = 500$ to 0.00530 at $n = 1000$ and remained similar at 0.00549 for $n = 2000$. The small increase between the last two settings is below the

corresponding across-seed standard deviations and should not be interpreted as deterioration. Four-dimensional energy distance decreased more consistently, from 0.001773 to 0.000587 and then 0.000420.

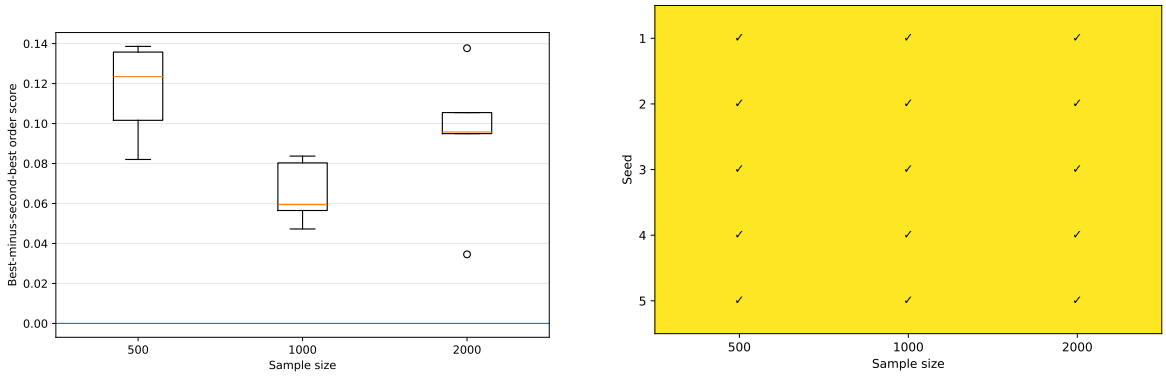
Across all 15 runs, the mean Kendall-$\tau$ MAE was 0.007026, the mean energy distance was 0.000927, and the median order-score margin was 0.094963. The finite-output, unit-interval, and all-six-orders-scored rates were each 1.0.




**(a)** Pairwise rank-dependence error.

**(b)** Four-dimensional energy distance.

**Figure 10.1:** Distributional fidelity by sample size. Points are means over five seeds; error bars show one sample standard deviation.




**(a)** Best-minus-second-best score margins.

**(b)** Recovery indicator by seed and sample size.

**Figure 10.2:** Parent-order diagnostics for the 15 final validation runs.

## 10.4 End-to-end integration

Each supported dataset generated 2,000 rows from a mixed-indegree graph with maximum indegree three and seed 42. All copula-space and original-scale values were finite, no missing values were present, and the audited copula-coordinate ranges were strictly within $(0, 1)$. These runs demonstrate successful integration of all four node cases and saved-artifact lookup.

They should not be used as headline realism comparisons. The graphs were constructed to exercise code paths rather than to match an accepted domain graph, and root nodes are sampled independently. A valid realism comparison must control the graph and its relationship to the calibration distribution.

## 10.5 What the evidence establishes

The final evidence supports the following statements:

- the repository implements graph-conditioned generation for maximum indegree three;
- exhaustive final summary files contain every combinatorial child/parent mechanism for the three datasets;
- the controlled four-dimensional mechanism selected the known generating order in the final 15-run design;
- generated controlled samples closely reproduced pairwise rank structure and had small energy distance; and
- saved libraries were successfully exercised in three end-to-end generation runs.

The evidence does not establish causal identification, intervention validity, universal order recovery, or superiority over unrelated synthetic-data methods.

Chapter 11

# Reproducibility and software quality

## 11.1 Configuration-driven provenance

The primary reproducibility unit is the dataset YAML file. It identifies the input table, file format, clipping constant, copula selection controls, order-selection controls, adaptive conditional controls, and output directories. Final results should therefore be interpreted together with `configs/datasets/*.yaml`, not only with the generated model files.

The report bundle records branch `feature/indegree-three` and commit `f5325c89060b5ba995bf21310f3aa9975dd9ace4`. The working-tree status captured when the bundle was assembled was not clean: `README.md` was modified and a temporary staging directory was untracked. Consequently, the commit hash identifies the checked-out code base, while the bundle itself is the authoritative snapshot for the modified documentation and final report inputs.

## 11.2 Seeds and deterministic splits

Random graph generation, graph-conditioned sampling, and controlled simulations all accept explicit seeds. Order selection uses a configured deterministic train/validation split. The parallel fitter keys checkpoints by canonical child/parent-set identifiers, making its task selection independent of completion order. These choices support repeatable workflows on a fixed software stack.

The project does not claim bitwise equivalence across operating systems, processors, C++ library builds, or dependency versions. Numerical optimization and parallel scheduling can affect floating-point details even when high-level outputs are stable.

## 11.3 Artifact manifests and Git LFS

The final three-parent archives are represented by one manifest per dataset plus a combined manifest and checksum list. Each per-dataset manifest records expected mechanisms, successful mechanisms, JSON file count, archive size, archive SHA-256, library SHA-256, and extraction target. The bundle's Git-LFS listing confirms that the three compressed archives were tracked, but the archives themselves were intentionally omitted from the report bundle to avoid duplicating large binary deliverables.

A fresh checkout must execute:

```
git lfs install
git lfs pull
git lfs ls-files
```

then extract the appropriate archive into the configured artifact directory. The repository

includes packaging and installation scripts, but a complete fresh-clone installation test was not part of the final accepted evidence.

## 11.4 Captured test evidence

The bundle contains the exact pytest output and package freeze. The final test record is:

**Captured software-test result**

`175 passed in 32.91s` exit code 0.

The separate release-check command failed at import time because `src/` was not on the Python import path in that captured invocation. This is documented rather than concealed. Since it did not execute its checks, it is not used as evidence for final success. The independent evidence audit reloaded the CSV and JSON files directly and recalculated the principal counts, domains, and aggregate metrics.

## 11.5 End-to-end reproduction checklist

A practical reproduction sequence is:

1. clone the repository and select the final branch or commit;
2. create a Python 3.11 environment and install `requirements.txt`;
3. pull Git-LFS objects and install or extract the precomputed libraries;
4. verify input table paths and dataset configurations;
5. run `python -m pytest -q`;
6. generate or supply a column-aligned DAG;
7. run the matching indegree generator with an explicit seed;
8. evaluate generated data with an explicit output directory; and
9. retain configuration, graph, seed, metadata, and commit hash with every result.

Complete commands appear in Chapter A.

## 11.6 Known reproducibility gaps

The final snapshot has four material gaps:

- no verified fresh-clone extraction-and-generation run is recorded;
- the generated metadata embeds absolute Windows paths;
- the configuration family list duplicates a family set constructed in Python rather than acting as the sole source of truth; and
- no lock file or container image fixes transitive dependency builds.

These are maintenance priorities rather than reasons to discard the verified statistical results.

Chapter 12

# Limitations and future extensions

## 12.1 Statistical limitations

**Local rather than unrestricted joint calibration.** The simulator composes local mechanisms selected by a supplied graph. If that graph omits an observed dependency, the generated distribution omits it by construction. Independent root sampling is the clearest example. The method should therefore be evaluated relative to the chosen graph, not assumed to reproduce the unrestricted calibration joint distribution.

**Empirical marginal extrapolation.** Empirical quantiles remain within the observed range. The model cannot generate genuinely new marginal extremes beyond the calibration support, and tail estimates are limited by finite rank resolution.

**Simplifying assumption at indegree three.** The adaptive quantile-binned model applies only to the final conditional edge of two-parent mechanisms. All conditional pair copulas in the three-parent mechanism are simplified. Extending non-simplified modelling to four-dimensional mechanisms is a substantial statistical extension because several conditioning sets and model-selection interactions must be handled.

**Finite candidate family set.** Selection is restricted to six families and rotations. Exact family recovery is imperfect even in controlled simulations, especially for high-degree Student copulas. A selected family should be treated as a useful parametric representation, not an identified physical mechanism.

**Order-selection evidence is controlled and small.** Perfect recovery in 15 controlled runs is encouraging but not universal. The experiment uses one designed D-vine family configuration and five seeds per sample size. Different families, weaker dependencies, near-equivalent orders, or misspecification may produce ambiguity.

## 12.2 Graph and causal limitations

The graph is supplied, not inferred. Calibration uses observational conditional distributions and does not justify edge direction, absence of hidden confounding, modular interventions, or counterfactual semantics. Supporting interventional generation would require explicit structural assumptions and a defined mechanism-replacement API; it is not a minor flag change.

The hard maximum indegree is three. Supporting four parents would require a five-dimensional local vine, $4! = 24$ candidate parent orders under the current exhaustive strategy, more pair-copula files per record, and new sampling and validation code. The combinatorial number of mechanisms would also grow to $d\binom{d-1}{4}$.

## 12.3 Data limitations

Only continuous numeric tables without missing values are accepted in the final workflows. Categorical, count, censored, or mixed variables would require discrete or mixed copula treatment and changes to pseudo-observations, h-function inputs, marginals, and tests. The three included datasets are calibration examples rather than a broad domain benchmark.

## 12.4 Computational limitations

Exhaustive precomputation scales combinatorially with dimension and maximum indegree. The final parallel runner addresses wall-clock execution on a multicore machine but does not change the number of fitted mechanisms. For larger $d$, an on-demand cache or graph-specific manifest would be more economical than exhaustive enumeration.

Model storage also grows rapidly: every three-parent record references six JSON pair-copula files. Compressed archives reduce distribution overhead, but raw extraction creates many small files.

## 12.5 Software-engineering limitations

- Dataset YAML files list candidate families, while the fitting code constructs the same family set explicitly. A future refactor should parse one canonical family registry from configuration.
- Absolute paths appear in some stored summaries and generation metadata. Loaders often recover by model basename, but artifact schemas should store project-relative paths.
- Pickle libraries are convenient but couple artifacts to Python class definitions. A versioned, language-neutral manifest schema would improve longevity.
- The repository includes several orchestration scripts with overlapping responsibilities. A package entry point and unified CLI would reduce maintenance burden.
- The final environment is recorded by `pip freeze`, but no locked environment or continuous-integration matrix is included in the supplied snapshot.

## 12.6 Evaluation limitations

The final controlled metrics emphasize rank dependence and four-dimensional energy distance. They do not evaluate downstream causal-learning performance, privacy, disclosure risk, intervention validity, utility for prediction, or full real-dataset graph fidelity. The end-to-end mixed graphs are integration tests, not controlled realism benchmarks.

## 12.7 Prioritized future work

1. **Minor engineering:** make family sets and model paths fully configuration-driven and relative; add a fresh-clone CI job that pulls LFS artifacts and generates a small table.

2. **Moderate algorithmic extension:** add graph-specific on-demand mechanism fitting with caching, avoiding exhaustive libraries when only a few graphs are needed.
3. **Substantial algorithmic extension:** develop and validate non-simplified three-parent conditional copulas with principled regularization across multidimensional conditioning coordinates.
4. **Research extension:** define intervention semantics and test whether locally calibrated observational mechanisms can be modified coherently under domain-specific causal assumptions.
5. **Evaluation extension:** compare downstream learning performance and graph-aware fidelity across controlled DAG families while holding graph, sample size, and seed sets fixed.

Chapter 13

# Conclusion

The completed Copula Causal Simulator is a modular continuous-data simulator that calibrates empirical marginals and local copula mechanisms from real tables, stores them as reusable libraries, and generates from a supplied DAG. Its central technical contribution is the extension from pair and two-parent mechanisms to explicit three-parent conditional D-vines. For every three-parent set, all six parent permutations are evaluated by held-out child-conditional log-likelihood, the selected order is refitted on all data, and conditional samples are obtained through a verified chain of inverse h-functions.

The final snapshot contains complete calibration summaries for Sachs, Diabetes, and Breast Cancer data: 166 unordered pair records, 1,515 two-parent mechanisms, and 4,140 three-parent mechanisms, all marked successful. Controlled three-parent validation recovered the generating order in all 15 accepted runs and produced small rank and energy discrepancies. Mixed-indegree end-to-end runs confirmed that saved artifacts can be combined with valid DAGs to generate finite copula-space and original-scale tables.

These results establish a functioning graph-conditioned statistical simulator with maximum indegree three. They do not establish causal discovery or intervention validity, and they do not remove the dependence of realism on the supplied graph. The most important next steps are artifact portability, configuration cleanup, on-demand calibration for larger problems, and statistically principled non-simplified models beyond two parents.

For a new developer, the stable conceptual division is: data loading and empirical marginals; reusable pair and D-vine calibration; graph validation; topological conditional sampling; and evaluation. The appendices provide commands, repository paths, artifact contracts, extension points, and the evidence audit needed to reproduce or continue the project.

Chapter A

# Complete reproduction commands

The commands below use Windows PowerShell syntax, matching the captured final environment. On Linux or macOS, activate `.venv/bin/activate`, replace backslashes with forward slashes, and use shell line continuations.

## A.1 Environment

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
python -m pip check
$env:PYTHONPATH = "$PWD\src"
```

## A.2 Retrieve precomputed artifacts

```
git lfs install
git lfs pull
git lfs ls-files
```

Inspect each `artifacts/precomputed_indegree3/*manifest.json` for its `extract_into` destination, verify the SHA-256 checksum, and extract the corresponding archive. The optional installer script automates this process when the archives are present.

## A.3 Rebuild base and two-parent libraries

```
python scripts/build_coupling_library.py ‘
  --config configs\datasets\sachs.yaml

python scripts/build_conditional_vine_library.py ‘
  --config configs\datasets\sachs.yaml ‘
  --parent-set-size 2
```

Repeat with the Diabetes and Breast Cancer configuration paths.

## A.4 Rebuild an exhaustive three-parent library

```
$env:OMP_NUM_THREADS = "1"
$env:MKL_NUM_THREADS = "1"
```

```
$env:OPENBLAS_NUM_THREADS = "1"
$env:NUMEXPR_NUM_THREADS = "1"

python -u scripts\run_indegree3_parallel.py `
  --config configs\datasets\sachs.yaml `
  --workers 8 `
  --checkpoint-every 1 `
  --restart
```

To resume after interruption, remove `-restart`. To retry records with failed status, add `-retry-failed`.

## A.5 Generate a bounded-indegree DAG

```
python scripts\generate_random_graphs.py `
  --num-graphs 1 `
  --num-nodes 11 `
  --edge-prob 0.25 `
  --max-indegree 3 `
  --seed 42 `
  --output-dir artifacts\graphs\sachs_example
```

## A.6 Generate synthetic data

```
python -u scripts\generate_from_graph_indegree3.py `
  --config configs\datasets\sachs.yaml `
  --graph artifacts\graphs\sachs_example\graph_0000_adjacency.csv `
  --n-samples 2000 `
  --seed 42 `
  --output-dir outputs\sachs_example
```

## A.7 Evaluate generated data

```
python scripts\evaluate_generated_data.py `
  --config configs\datasets\sachs.yaml `
  --generated-x outputs\sachs_example\generated_x.csv `
  --graph artifacts\graphs\sachs_example\graph_0000_adjacency.csv `
  --output-dir reports\evaluation\sachs_example `
  --tail-probability 0.05
```

## A.8 Controlled three-parent validation

```
python -u scripts\synthetic_validation\run_indegree3_validation.py `
  --n 500 `
  --seed 1 `
```

```
  --output-dir reports\validation\n500_seed1
```

The accepted aggregate experiment repeats this for $n \in \{500, 1000, 2000\}$ and seeds 1 through 5.

## A.9 Tests

```
python -m pytest -q
python -m pytest tests\test_indegree_three.py -q
```

## A.10 Package rebuilt artifacts

```
python -u scripts\package_indegree3_artifacts.py
sha256sum artifacts/precomputed_indegree3/*.tar.gz
```

Track compressed archives through Git LFS rather than normal Git objects.

Chapter B

# Repository map and artifact schemas

## B.1 Principal source modules

| Path | Responsibility |
|---|---|
| src/copula_causal_sim/config.py | YAML loading and project-relative configuration handling. |
| src/copula_causal_sim/data/loaders.py | Tabular input parsing, column selection, numeric and missing-value validation. |
| src/copula_causal_sim/copulas/marginals.py | Rank pseudo-observations and empirical marginal libraries. |
| src/copula_causal_sim/copulas/paircopula_library.py | Pair-family controls, all-pairs fitting, diagnostics, JSON and pickle metadata. |
| src/copula_causal_sim/copulas/conditional_vine_library.py | Explicit two-parent D-vines, order selection, adaptive quantile-bin conditional models. |
| src/copula_causal_sim/copulas/non_simplified.py | Quantile-bin fitting, selection, and diagnostics for non-simplified conditional pairs. |
| src/copula_causal_sim/copulas/conditional_vine_indegree3.py | Explicit four-dimensional D-vines, six-order selection, conditional log-likelihood and sampling. |
| src/copula_causal_sim/graphs/dag.py | Adjacency contract, degree queries, topological sorting and validation. |
| src/copula_causal_sim/graphs/random_graphs.py | Seeded random bounded-indegree DAG sampling. |
| src/copula_causal_sim/generators/indegree_one.py | Root and one-parent graph-conditioned generation. |
| src/copula_causal_sim/generators/indegree_two.py | Mixed zero/one/two-parent generation and adaptive conditional-bin sampling. |
| src/copula_causal_sim/generators/indegree_three.py | Mixed zero/one/two/three-parent generation. |
| src/copula_causal_sim/evaluation/realism.py | Marginal, correlation, graph-pair, and tail diagnostics. |
| src/copula_causal_sim/synthetic/ | Controlled synthetic mechanisms and recovery-report utilities. |

## B.2 Important scripts

| Script | Use |
|---|---|
| scripts/build_coupling_library.py | Build dataset summary, pseudo-observations, marginals, and all pair copulas. |
| scripts/build_conditional_vine_library.py | Build exhaustive two- or three-parent conditional libraries serially. |
| scripts/run_indegree3_parallel.py | Checkpointed multicore exhaustive three-parent fitting. |
| scripts/generate_random_graphs.py | Write random bounded-indegree adjacency matrices and metadata. |
| scripts/generate_from_graph_indegree1.py | Generate for DAGs of maximum indegree one. |
| scripts/generate_from_graph_indegree2.py | Generate for DAGs of maximum indegree two. |
| scripts/generate_from_graph_indegree3.py | Generate for DAGs of maximum indegree three. |

| Script | Use |
|---|---|
| `scripts/evaluate_generated_data.py` | Compare generated original-scale data with calibration data. |
| `scripts/package_indegree3_artifacts.py` | Validate, compress, checksum, and manifest final three-parent artifacts. |
| `scripts/validate_final_indegree3_release.py` | Check final libraries, end-to-end outputs, and multi-seed files when import paths are configured. |

## B.3 Pair-copula record

A final pair record identifies the two variables and fitted order, selected family and rotation, parameter array, number of parameters, log-likelihood, AIC, BIC, empirical Kendall and Spearman dependence, tail diagnostics, JSON model file, and status/error. The pickle library maps an unordered pair key to this record.

## B.4 Two-parent record

A two-parent record includes child, canonical parent set, selected order, both candidate scores and statuses, selected conditional model type, bin metadata when applicable, three pair-model paths, fit summaries, diagnostics, and status. The lookup key is the child plus an unordered parent set.

## B.5 Three-parent record

A three-parent record contains:

- child and canonical three-parent set;
- selected order and all six candidate scores/statuses/errors;
- requested and applied order-selection method;
- split sizes, seed, and minimum validation rows;
- six pair-copula JSON paths, families, rotations, and parameters;
- conditional and joint fit summaries; and
- status/error fields.

The final archive manifest adds total and successful mechanism counts, JSON model count, archive and library checksums, and extraction destination.

Chapter C

# Developer continuation guide

## C.1 Add a calibration dataset

1. Prepare a numeric CSV, TSV, or whitespace-delimited table. Remove targets that should not be modelled jointly.
2. Add a YAML file under `configs/datasets/` with a unique `dataset_id`, relative `path`, supported `file_type`, data policy, copula controls, and distinct output directories.
3. If preparation is reproducible from an external package, add a script analogous to `prepare_sklearn_diabetes.py` and write provenance metadata.
4. Run the coupling-library builder and inspect `dataset_summary.csv` and `paircopula_summary.csv`.
5. Build two- and three-parent libraries as required.
6. Add loader, generator, and end-to-end tests for the new column set.

Never reuse an artifact directory across dataset versions; column identity is part of the model schema.

## C.2 Add or change a copula family

The final family registry is constructed in `paircopula_library.py::build_fit_controls` and `conditional_vine_library.py::build_bicop_fit_controls`. Add the corresponding `pyvinecopulib.BicopFamily` value in both locations, update validation configuration, and add controlled recovery scenarios. Also reconcile the YAML `families` field so that configuration and code cannot diverge.

A new family must support the operations used by the generators: density, h-functions, inverse h-functions, JSON serialization, and parameter-to-$\tau$ conversion where diagnostics require it.

## C.3 Change selection controls

Dataset-level controls live under `copula:` in YAML. The serial conditional builder exposes command-line overrides for selection criterion, order strategy, validation fraction, seeds, minimum validation rows, adaptive conditional strategy, bin candidates, minimum bin size, and score-improvement threshold. When changing controls:

1. write to a new artifact directory;
2. preserve the exact config with the results;
3. rerun controlled synthetic validation; and

4. avoid comparing libraries calibrated under different controls without explicit labels.

## C.4 Add a graph

Construct a headerless square binary CSV in dataset column order. Validate it with `graphs/dag.py::validate_dag`; confirm dimension, acyclicity, self-loop absence, and maximum indegree. Generation fails clearly if a required mechanism is missing. For reusable graph collections, save the adjacency and a metadata JSON containing variable order, edge list, seed, and graph summary.

## C.5 Support a larger parent set

A principled indegree-four extension requires more than changing a constant:

1. define an explicit five-dimensional local vine with child fixed last;
2. implement coordinate transforms, conditional log-likelihood, and inverse sampling;
3. decide whether all 24 parent orders are evaluated;
4. create a record/library schema and canonical parent-set lookup;
5. integrate the new sampler into a maximum-indegree-four generator;
6. add serialization, graph-bound, range, reproducibility, and known-mechanism tests; and
7. design controlled order and distributional validation.

Exhaustive storage grows as $d\binom{d-1}{4}$ records. On-demand fitting and caching should be considered before full precomputation.

## C.6 Add an evaluation metric

Implement reusable computation in `evaluation/realism.py`; keep file I/O and CLI parsing in `scripts/evaluate_generated_data.py`. Define the estimator, direction of improvement, required representation ($X$ or $U$), graph dependence, and aggregation. Add unit tests with analytically simple data, then add a stable output filename and summary key.

## C.7 Run a new benchmark

Use the existing benchmark scripts only after checking their current CLI and output schema. A defensible benchmark fixes dataset, graph source, edge density or exact graph, sample size, seeds, calibration library, and evaluation settings. Store one metadata record per run and aggregate only comparable runs.

## C.8 Diagnose missing or incompatible artifacts

Check in this order:

1. dataset ID and exact column order in YAML and library metadata;
2. presence of marginal, pair, and required conditional library pickles;
3. existence of the referenced JSON model basename under the expected model directory;
4. child plus canonical parent-set key in the summary/library;
5. graph maximum indegree and dimension;
6. Git-LFS archive download and extraction checksum;
7. compatibility of pickle classes with the checked-out source commit.

Absolute paths in old records should not be blindly recreated; prefer project-relative lookup by configured model directory and basename.

## C.9 Test a new mechanism

At minimum, test finite outputs, unit-interval bounds, deterministic fixed-seed behaviour, sensitivity to a changed seed, serialization round trips, both pair orientations, all supported parent orders, graph integration, and recovery from a controlled known model. Statistical validation should include a dependence-strength metric and a multivariate discrepancy, not only successful execution.

# References


[1] Kjersti Aas, Claudia Czado, Arnoldo Frigessi, and Henrik Bakken. "Pair-Copula Constructions of Multiple Dependence". In: *Insurance: Mathematics and Economics* 44.2 (2009), pp. 182–198. DOI: 10.1016/j.insmatheco.2007.02.001.

[2] Tim Bedford and Roger M. Cooke. "Probability Density Decomposition for Conditionally Dependent Random Variables Modeled by Vines". In: *Annals of Mathematics and Artificial Intelligence* 32 (2001), pp. 245–268. DOI: 10.1023/A:1016725902970.

[3] Jeffrey Dißmann, Eike Christian Brechmann, Claudia Czado, and Dorota Kurowicka. "Selecting and Estimating Regular Vine Copulae and Application to Financial Returns". In: *Computational Statistics & Data Analysis* 59 (2013), pp. 52–69. DOI: 10.1016/j.csda.2012.08.010.

[4] Maurice G. Kendall. "A New Measure of Rank Correlation". In: *Biometrika* 30.1–2 (1938), pp. 81–93. DOI: 10.1093/biomet/30.1-2.81.

[5] Roger B. Nelsen. *An Introduction to Copulas*. 2nd ed. New York: Springer, 2006. DOI: 10.1007/0-387-28678-0.

[6] Karen Sachs, Omar Perez, Dana Pe'er, Douglas A. Lauffenburger, and Garry P. Nolan. "Causal Protein-Signaling Networks Derived from Multiparameter Single-Cell Data". In: *Science* 308.5721 (2005), pp. 523–529. DOI: 10.1126/science.1105809.

[7] Gideon Schwarz. "Estimating the Dimension of a Model". In: *The Annals of Statistics* 6.2 (1978), pp. 461–464. DOI: 10.1214/aos/1176344136.

[8] scikit-learn developers. *load_breast_cancer: Breast Cancer Wisconsin Dataset Loader*. 2026. URL: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html (visited on 07/13/2026).

[9] scikit-learn developers. *load_diabetes: Diabetes Dataset Loader*. 2026. URL: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html (visited on 07/13/2026).

[10] Abe Sklar. "Fonctions de répartition à n dimensions et leurs marges". In: *Publications de l'Institut de Statistique de l'Université de Paris* 8 (1959), pp. 229–231.

[11] W. Nick Street, William H. Wolberg, and Olvi L. Mangasarian. "Nuclear Feature Extraction for Breast Tumor Diagnosis". In: *Biomedical Image Processing and Biomedical Visualization*. Vol. 1905. SPIE, 1993, pp. 861–870. DOI: 10.1117/12.148698.

[12] Gábor J. Székely and Maria L. Rizzo. "Energy Statistics: A Class of Statistics Based on Distances". In: *Journal of Statistical Planning and Inference* 143.8 (2013), pp. 1249–1272. DOI: 10.1016/j.jspi.2013.03.018.

[13] vinecopulib contributors. *pyvinecopulib: A Python Library for Vine Copula Models*. Version 0.7.6. 2026. URL: https://github.com/vinecopulib/pyvinecopulib (visited on 07/13/2026).